VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Experiment No. 02

Aim: - Write a simple Java programs based on basic Operators and expressions.

Software Requirement: - Java Development Kit (JDK), Integrated Development Environment (IDE) (NetBeans), Java Runtime Environment (JRE).

Theory: -

Java supports a rich set of operators. We have already used several of them, such as =, +, –, and *. An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations. Operators are used in programs to manipulate data and variables. They usually form a part of mathematical or logical expressions.

Java operators can be classified into a number of related categories as below:

1. Arithmetic Operators
2. Increment & Decrement Operators
3. Relational Operators
4. Bitwise Operators
5. Logical Operators
6. Assignment Operators

1. Arithmetic Operators

Arithmetic operators are used to construct mathematical expressions as in algebra. Java provides all the basic arithmetic operators. These operators involve the mathematical operators that can be used to perform various simple or advanced arithmetic operations on the primitive data types referred to as the operands. These operators consist of various unary and binary operators that can be applied on a single or two operands.

Let's look at the various operators that Java has to provide under the arithmetic operators.

Sr. No.	Operators	Explanation	Syntax	Example
1	**Addition(+)**	This operator is a binary operator and is used to add two operands.	num1 + num2	num1 = 10, num2 = 20 sum = num1 + num2 = 30
2	**Subtraction(-)**	This operator is a binary operator and is used to subtract two operands.	num1 - num2	num1 = 20, num2 = 10 sub = num1 - num2 = 10
3	**Multiplication(*)**	This operator is a binary operator and is used to multiply two operands.	num1 * num2	num1 = 20, num2 = 10 mult = num1 * num2 = 200
4	**Division(/)**	This is a binary operator that is used to divide the first operand(dividend) by the second operand(divisor) and give the quotient as a result.	num1 / num2	num1 = 20, num2 = 10 div = num1 / num2 = 2
5	**Modulus(%):**	This is a binary operator that is used to return the remainder when the first operand(dividend) is divided by the second operand(divisor).	num1 % num2	num1 = 5, num2 = 2 mod = num1 % num2 = 1

2. Increment & Decrement Operators

Java has two very useful operators not generally found in many other languages. These are the increment and decrement operators. The increment operator increases its operand by one. The decrement operator decreases its operand by one.

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

They are used to increment, decrement, or negate a value.

- `-` , Negates the value.
- `+` , Indicates a positive value (automatically converts `byte`, `char`, or `short` to `int`).
- `++` , Increments by 1.
  - **Post-Increment**: Uses value first, then increments.
  - **Pre-Increment**: Increments first, then uses value.
- `--` , Decrements by 1.
  - **Post-Decrement**: Uses value first, then decrements.
  - **Pre-Decrement**: Decrements first, then uses value.

Example

x = x + 1;

The above statement can be rewritten by using the increment operator

x++;

Similarly, the statement

x = x – 1

is equivalent to

x--;

3. Relational Operators

Java Relational Operators are a bunch of binary operators used to check for relations between two operands, including equality, greater than, less than, etc. They return a boolean result after the comparison and are extensively used in looping statements as well as conditional if-else statements and so on.

The general format of representing relational operators are:

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Operator	Meaning	Syntax
==	Equal to	(A == B)
!=	Not equal to	(A != B)
>	Greater than	(A > B)
<	Less than	(A < B)
>=	Greater than or equal to	(A >= B)
<=	Less than or equal to	(A <= B)

Syntax:

```
variable1 relation_operator variable2
```

Let us look at each one of the relational operators in Java:

Sr. No.	Operators	Explanations	Syntax	Example
1	**'Equal to' operator (==)**	This operator is used to check whether the two given operands are equal or not. The operator returns true if the operand at the left-hand side is equal to the right-hand side, else false.	var1 == var2	var1 = "JDCollege" var2 = 20 var1 == var2 results in false
2	**'Not equal to' Operator(!=)**	This operator is used to check whether the two given operands are equal or not. It functions opposite to that of the equal-to-operator. It returns true if the operand at the left-hand side is not equal to the right-hand side, else false.	var1 != var2	var1 = "GeeksforGeeks" var2 = 20 var1 != var2 results in true
3	**'Greater than' operator(>)**	This checks whether the first operand is greater than the second operand or not. The operator returns true when the operand at the left-hand side is greater than the right-hand side.	var1 > var2	var1 = 30 var2 = 20 var1 > var2 results in true

4	**'Less than' Operator(<)**	This checks whether the first operand is less than the second operand or not. The operator returns true when the operand at the left-hand side is less than the right-hand side. It functions opposite to that of the greater-than operator.	var1 < var2	var1 = 10 var2 = 20 var1 < var2 results in true
5	**Greater than or equal to (>=)**	This checks whether the first operand is greater than or equal to the second operand or not. The operator returns true when the operand at the left-hand side is greater than or equal to the right-hand side.	var1 >= var2	var1 = 20 var2 = 20 var3 = 10 var1 >= var2 results in true var2 >= var3 results in true
6	**Less than or equal to (<=)**	This checks whether the first operand is less than or equal to the second operand or not. The operator returns true when the operand at the left-hand side is less than or equal to the right-hand side.	var1 <= var2	var1 = 10 var2 = 10 var3 = 9 var1 <= var2 results in true var2 <= var3 results in false

4. Bitwise Operators

Java has a distinction of supporting special operators known as bitwise operators for manipulation of data at values of bit level. These operators are used for testing the bits, or shifting them to the right or left. Bitwise operators may not be applied to float or double.

Example

Bitwise operator works on bits and performs bit-by-bit operation. Assume if a = 60; and b =

13; now in binary format, value of a & b will be

a = 0011 1100

b = 0000 1101

a&b = 0000 1100

a|b = 0011 1101

a^b = 0011 0001

~a = 1100 0011

Operator	Description	Syntax
&	Bitwise AND	(A & B)
|	Bitwise OR	(A | B)
^	Bitwise exclusive OR	(A ^ B)
~	Bitwise unary NOT	(~A).
<<	Shift left	A << 2
>>	Shift right	A >> 2
>>>	Shift right zero fill	A >>>2 will give 15 which is 0000 1111

5. Logical Operators

Logical operators are used to perform logical "AND", "OR" and "NOT" operations, i.e. the function similar to AND gate and OR gate in digital electronics. They are used to combine two or more conditions/constraints or to complement the evaluation of the original condition under particular consideration. One thing to keep in mind is, while using AND operator, the second condition is not evaluated if the first one is false. Whereas while using OR operator, the second condition is not evaluated if the first one is true, i.e. the AND and OR operators have a short-circuiting effect. Used extensively to test for several conditions for making a decision.

- **AND Operator** (**&&**) – if(a && b) [if true execute else don't]
- **OR Operator** (||) – if(a || b) [if one of them is true to execute else don't]

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

- **NOT Operator** (**!**) – !(a<b) [returns false if a is smaller than b]

Example For Logical Operator in Java

Here is an example depicting all the operators where the values of variables a, b, and c are kept the same for all the situations.

a = 10, b = 20, c = 30

For AND operator:

Condition 1: c > a
Condition 2: c > b
Output:
True [Both Conditions are true]

For OR Operator:

Condition 1: c > a
Condition 2: c > b
Output:
True [One of the Condition if true]

For NOT Operator:

Condition 1: c > a
Condition 2: c > b
Output:
False [Because the result was true and NOT operator did it's opposite]

Let's look at each of the Logical operators and how they operate:

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Sr. No.	Operators	Explanation	Syntax	Example

1	**Logical 'AND' Operator (&&)**	This operator returns true when both the conditions under consideration are satisfied or are true. If even one of the two yields false, the operator results false. In Simple terms, ***cond1 && cond2 returns true when both cond1 and cond2 are true (i.e. non-zero***).	condition1 **&&** condition2	a = 10, b = 20, c = 20
				
				condition1: a < b
				condition2: b == c
				
				if(condition1 && condition2)
				d = a + b + c
				
				// Since both the conditions are true
				d = 50.

2	**Logical 'OR' Operator (||)**	This operator returns true when one of the two conditions under consideration is satisfied or is true. If even one of the two yields true, the operator results true. To make the result false, both the constraints need to return false.	condition1 || condition2	a = 10, b = 20, c = 20 **condition1:** a < b **condition2:** b > c **if(condition1 || condition2)** d = a + b + c **// Since one of the condition is true** d = 50.

3	**Logical 'NOT' Operator (!)**	Unlike the previous two, this is a unary operator and returns true when the condition under consideration is not satisfied or is a false condition. Basically, if the condition is false, the operation returns true and when the condition is true, the operation returns false.	**!**(condition)	a = 10, b = 20
				
				!(a<b) // returns false
				
				!(a>b) // returns true

6. Assignment Operators

These operators are used to assign values to a variable. The left side operand of the assignment operator is a variable, and the right side operand of the assignment operator is a value. The value on the right side must be of the same data type of the operand on the left side. Otherwise, the compiler will raise an error. This means that the assignment operators have right to left associativity, i.e., the value given on the right-hand side of the operator is assigned to the variable on the left. Therefore, the right-hand side value must be declared before using it or should be a constant. The general format of the assignment operator is,

variable operator value;

Let's look at each of the assignment operators and how they operate:

Sr. No.	Operators	Explanation	Syntax	Examples
1	(=) operator	This is the most straightforward assignment operator, which is used to assign the value on the right to the variable on the left. This is the basic definition of an assignment operator and how it functions.	num1 = num2;	a = 10; ch = 'y';
2	(+=) operator	This operator is a compound of '+' and '=' operators. It operates by adding the current value of the variable on the left to the value on the right and then assigning the result to the operand on the left.	num1 += num2;	a += 10 This means, a = a + 10
3	(-=) operator	This operator is a compound of '-' and '=' operators. It operates by subtracting the variable's value on the right from the current value of the variable on the left and then assigning the result to the operand on the left.	num1 -= num2;	a -= 10 This means, a = a - 10
4	(*=) operator	This operator is a compound of '*' and '=' operators. It operates by multiplying the current value of the variable on the left to the value on	num1 *= num2;	a *= 10 This means, a = a * 10

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

		the right and then assigning the result to the operand on the left.		
5	. (/=) operator	This operator is a compound of '/' and '=' operators. It operates by dividing the current value of the variable on the left by the value on the right and then assigning the quotient to the operand on the left.	num1 /= num2;	a /= 10 This means, a = a / 10
6	(%=) operator	This operator is a compound of '%' and '=' operators. It operates by dividing the current value of the variable on the left by the value on the right and then assigning the remainder to the operand on the left.	num1 %= num2;	a %= 3 This means, a = a % 3

Programs

1. Arithmetic Operators

```
public class ArithmDemo
{
public static void main (String[] args)
{
int add = 2 + 4;
System.out.println( "Addition : 2 + 4 = " + add);
int subtract = 10 - 3;
System.out.println("Subtraction : 10 - 3 = " + subtract);
int divide = 6/2;
System.out.println("Division : 6 / 2 = " + divide);
int multiply = 5 * 5;
System.out.println("Muitiplication : 5 * 5 = " + multiply);
int modulus = 7 % 2;
System.out.println("Modulus : 7 % 2 = " + modulus);
}
}
```

Output:

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

```
Addition : 2 + 4 = 6
Subtraction : 10 - 3 = 7
Division : 6 / 2 = 3
Muitiplication : 5 * 5 = 25
Modulus : 7 % 2 = 1
```

2. Increment & Decrement Operators

```
class Incre {
    // main function
  public static void main(String[] args)
  {
     // Interger declared
     int a = 10;
     int b = 10;


     System.out.println("Postincrement : " + (a++));
     System.out.println("Preincrement : " + (++a));

     System.out.println("Postdecrement : " + (b--));
     System.out.println("Predecrement : " + (--b));
  }
}
```

Output:

```
Postincrement : 10
Preincrement : 12
Postdecrement : 10
Predecrement : 8
```

3. Relational Operators

```
class Relational {
   // main function
```

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

```
    public static void main(String[] args)
    {
        // Comparison operators
        int a = 10;
        int b = 3;
        int c = 5;

        System.out.println("a > b: " + (a > b));
        System.out.println("a < b: " + (a < b));
        System.out.println("a >= b: " + (a >= b));
        System.out.println("a <= b: " + (a <= b));
        System.out.println("a == c: " + (a == c));
        System.out.println("a != c: " + (a != c));
    }
}
```

Output:

```
a > b: true
a < b: false
a >= b: true
a <= b: false
a == c: false
a != c: true
```

4. Bitwise Operators

```
class Geeks
{
    public static void main(String[] args)
    {
        // Bitwise operators
        int d = 0b1010;
        int e = 0b1100;

        System.out.println("d & e : " + (d & e));
        System.out.println("d | e : " + (d | e));
        System.out.println("d ^ e : " + (d ^ e));
        System.out.println("~d : " + (~d));
        System.out.println("d << 2 : " + (d << 2));
        System.out.println("e >> 1 : " + (e >> 1));
        System.out.println("e >>> 1 : " + (e >>> 1));
```

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

```
    }
}
```

Output:

```
d & e : 8
d | e : 14
d ^ e : 6
~d : -11
d << 2 : 40
e >> 1 : 6
e >>> 1 : 6
```

5. Logical Operators

```
class Logical {

    // Main Function
   public static void main (String[] args) {

      // Logical operators
      boolean x = true;
      boolean y = false;

      System.out.println("x && y: " + (x && y));
      System.out.println("x || y: " + (x || y));
      System.out.println("!x: " + (!x));
   }
}
```

Output:

```
x && y: false
x || y: true
!x: false
```

6. Assignment Operators

```
class Assignment {
   // Main Function
   public static void main(String[] args)
   {

      // Assignment operators
```

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

```
        int f = 7;
        System.out.println("f += 3: " + (f += 3));
        System.out.println("f -= 2: " + (f -= 2));
        System.out.println("f *= 4: " + (f *= 4));
        System.out.println("f /= 3: " + (f /= 3));
        System.out.println("f %= 2: " + (f %= 2));
    }
}
```

Output:

```
f += 3: 10
f -= 2: 8
f *= 4: 32
f /= 3: 10
f %= 2: 0
```

Conclusion:

Hence, thus we perform simple Java programs based on basic Operators and expressions Successfully.